



3D компьютерное зрение и графика

Лекция 1. Введение. 3D-представления

Жемчужников Дмитрий Сергеевич

Лекции 1–5

Введение и 3D-представления

Геометрия: $LA \rightarrow SO(3)$

Камера и калибровка

Многовидовая геометрия

Matching и SfM

Лекции 6–10

Растеризация, графический
конвейер

Трассировка лучей

Освещение

Дифференцируемый
рендеринг I–II

Лекции 11–15

Оценка глубины

Многовидовая реконструкция

Семантическая
реконструкция

Advanced camera

SLAM



Дмитрий
Жемчужников



Сослан
Кабисов



Олег
Павлов



Максим
Елистратов



Владимир
Фролов



Альберт
Гарифуллин



Максим
Колодяжный



Никита
Дроздов





Дмитрий
Жемчужников

PhD университета Гренобль-Альпы, научный сотрудник Центра ИИ МГУ и AIRI, руководитель команды по CAD реконструкции. 1-5 лекции



Олег
Павлов

Аспирант Физического Факультета МГУ (практически к.н.). Раньше занимался ИИ для биофизики. Сейчас разрабатывает сети для облаков точек. 2 семинар.



Сослан
Кабисов

Магистр МФТИ, долго работал в продовом CV, сотрудник Сбера (по факту AIRI), главный разработчик методов image2cad. 4, 5 семинары



Максим
Елистратов

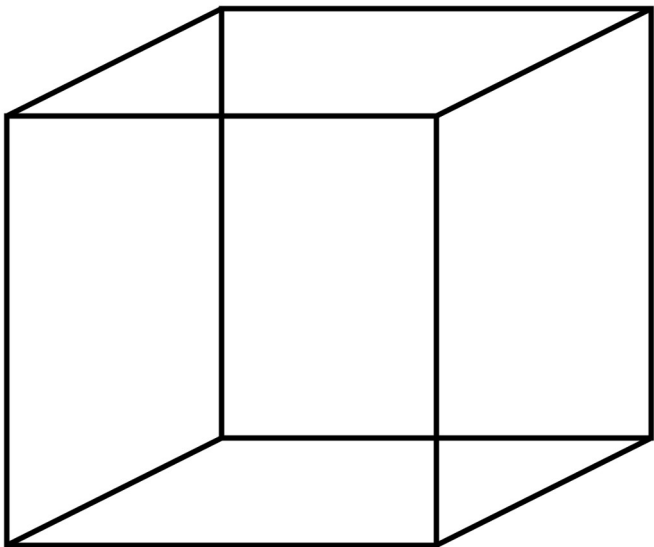
Аспирант Факультета ИИ МГУ (1 год). Имеет опыт работы инженера в авиастроении. Автор эволюционного пайплайна для генерации данных. 1, 3 семинары.



- 15 лекций и 15 семинаров, по одному семинару к каждой лекции
- По 2 домашних задания на каждый блок, всего 6 дз
- Устный экзамен



Как мы воспринимаем 3D-пространство



BenFrantzDale, CC BY-SA 3.0, Wikimedia Commons



Skagedal, public domain, Wikimedia Commons · маска Бьорна Борга, опыты в Уппсальском университете



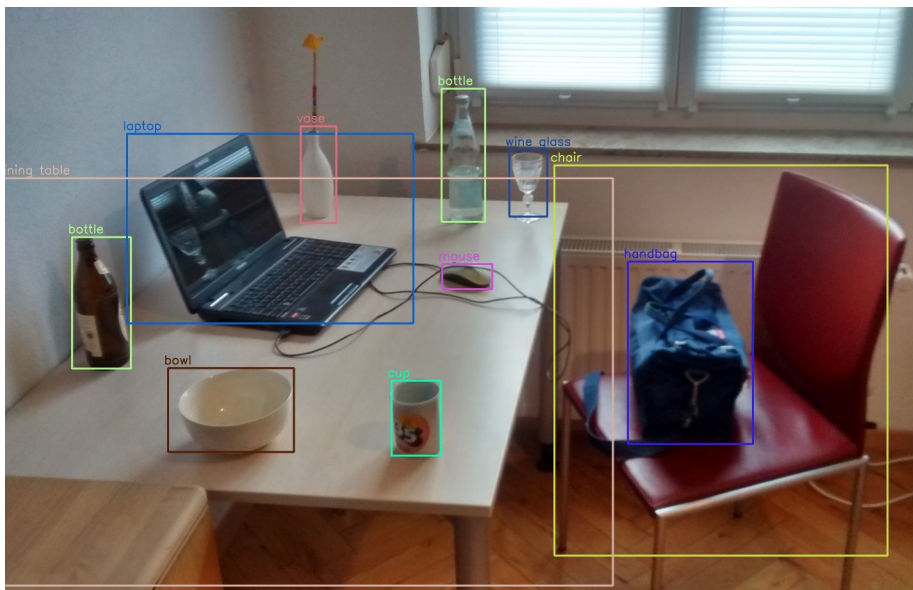
Одна картинка — два прочтения

- На сетчатке — двумерная проекция; трёхмерная сцена это реконструкция, а не измерение
- Мозг доопределяет её приорами: два глаза, движение, затенение, перспектива, перекрытия

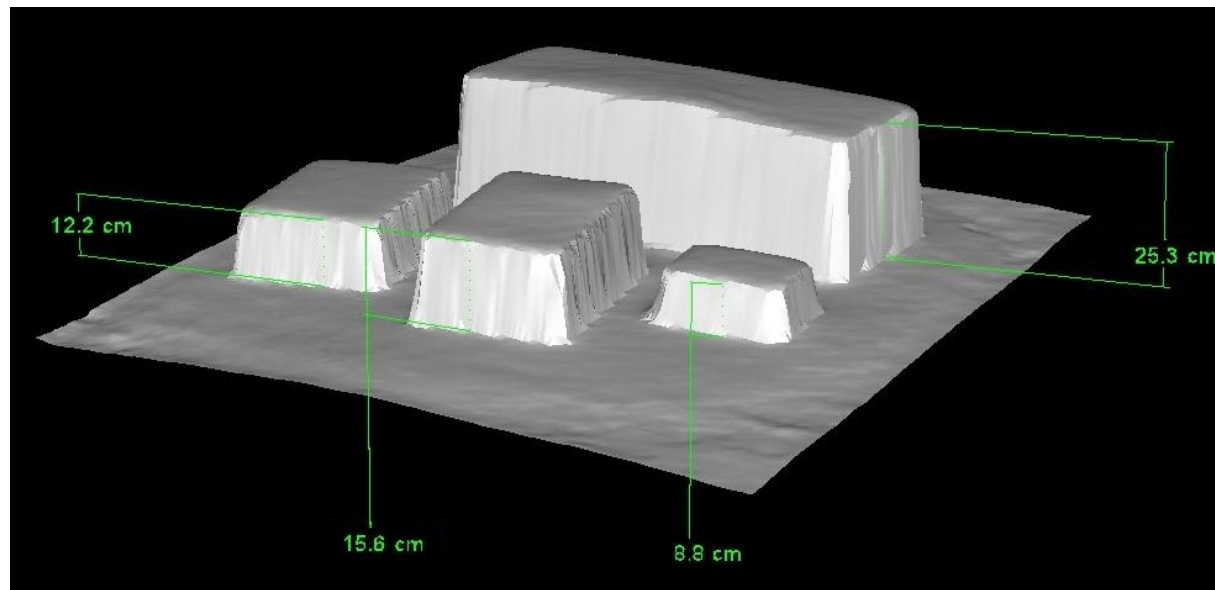
Маска вогнутая — но мозг видит выпуклое лицо

Где приор не выполняется — там иллюзия: и у мозга, и у алгоритма





MTheiler, CC BY-SA 4.0, Wikimedia Commons · детекция YOLOv3 (COCO)



Captaindistance, CC BY 3.0, Wikimedia Commons · времяпролётная камера

«Что это?» «Где на картинке?»

- класс: cup, laptop, bowl, chair
- рамка на картинке — ответ в пикселях
- это параллельный курс CV

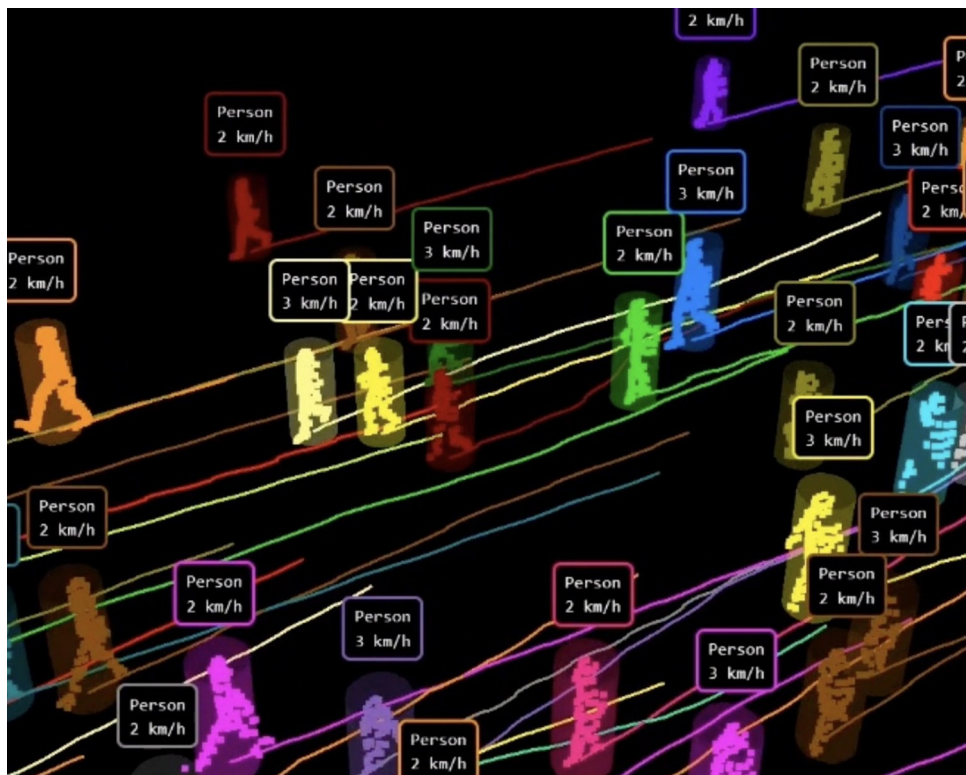
«Где» спрашивают оба курса — разница в том, в пикселях ответ или в метрах

«Где в пространстве?» «Какого размера?»

- положение и расстояние — ответ в метрах
- размеры 12,2 и 25,3 см — сверяет рулетка
- это наш курс



Почему одного семейства мало



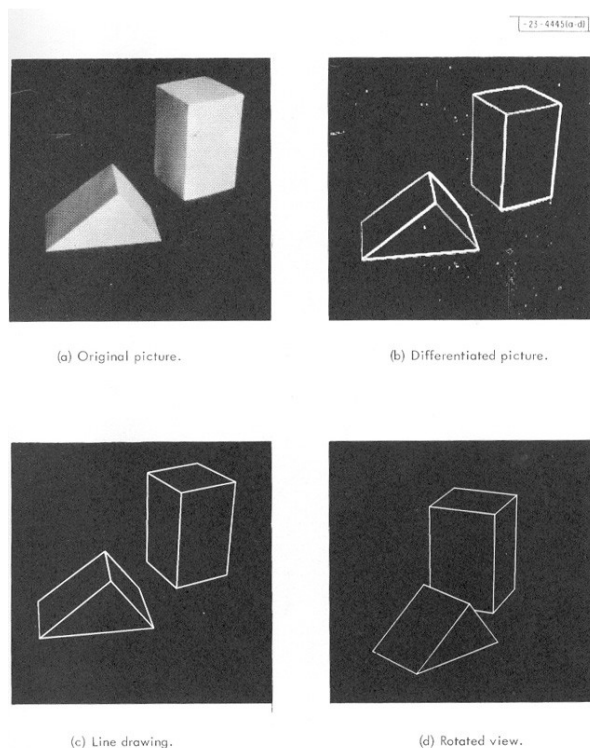
Saxotango, CC BY-SA 4.0, Wikimedia Commons · лидарный мониторинг людей

- Роботу метка «кружка» бесполезна: нужно, где именно и какого она размера
- Автомобилю мало «впереди пешеход»: нужны расстояние, скорость, габариты
- Шлему AR нужна геометрия комнаты — иначе виртуальный объект не перекроется мебелью
- На картинке у каждого объекта есть и класс, и скорость, и траектория — это и есть объединение двух семейств

3D-зрение = метрическое зрение + семантическое зрение

Не «здесь машина», а «машина длиной 4,5 м в семи метрах впереди, движется под углом 15°»





Три уровня Марра, 1982

1. Первичный эскиз

края, штрихи, пятна на изображении

2. 2½-мерный эскиз

глубина и ориентация видимых поверхностей
в системе координат наблюдателя

3. Трёхмерная модель

форма объекта в его собственных координатах

L. G. Roberts. Machine Perception of Three-Dimensional Solids. PhD thesis, MIT, 1963

По фотографии — рёбра, по рёбрам — 3D-
модель

Плоское зрение — более поздняя специализация; к 2020-м маятник качнулся обратно

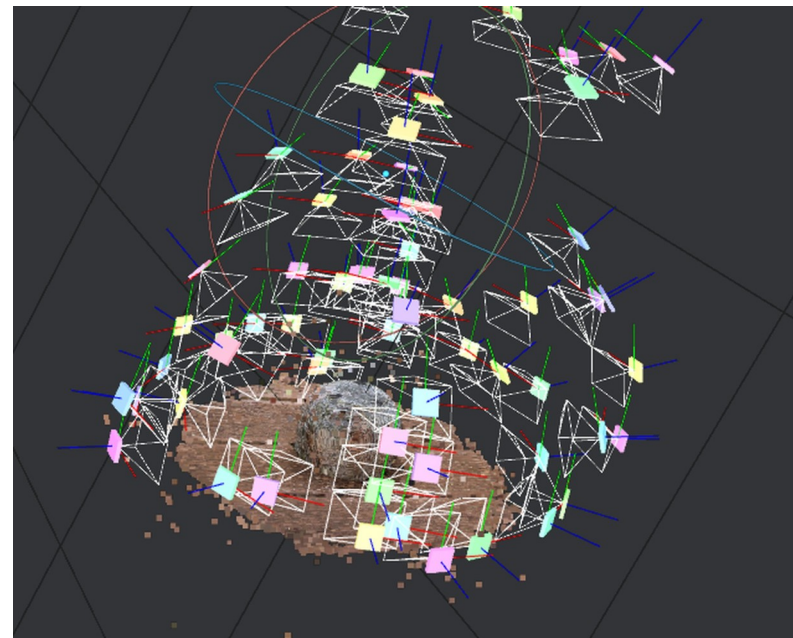




KaiaVintr, CC BY-SA 4.0, Wikimedia Commons · Blender Cycles, path tracing

Прямая задача: сцена → изображение

- задали форму, материалы, свет и камеру
- физика известна — картинка получается одна
- это графика: лекции 6–8



Colin Kranz, CC BY-SA 4.0, Wikimedia Commons · фотограмметрия в Meshroom

Обратная задача: изображения → сцена

- по снимкам восстановлены и форма, и позы камер
- решений бесконечно много — нужен дополнительный источник знания
- это наш курс: лекции 4–5, 12



Прямая и обратная задача: схема

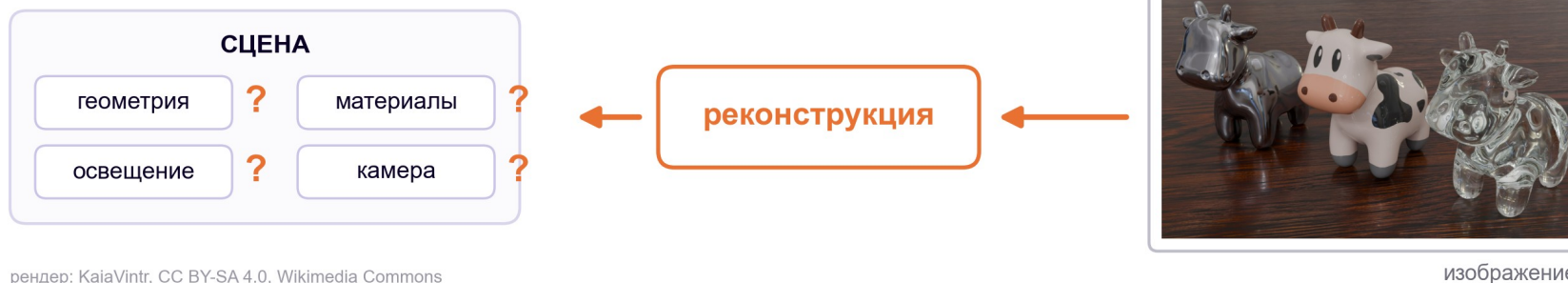
Прямая задача: сцена → изображение

физика известна — ответ единственный



Обратная задача: изображение → сцена

решений бесконечно много — нужен дополнительный источник знания



Неизвестны все четыре группы сразу: уравнений меньше, чем неизвестных

задача не «трудная», а некорректно поставленная: выбрать один ответ можно только по знанию, которого в изображении нет





настоящая трещина в леднике



краска на ровном асфальте, снято с нужной точки

Существует — да

объяснений не ноль, а слишком много

Единственно — нет

одному изображению отвечает
бесконечно много сцен

Устойчиво — не всегда

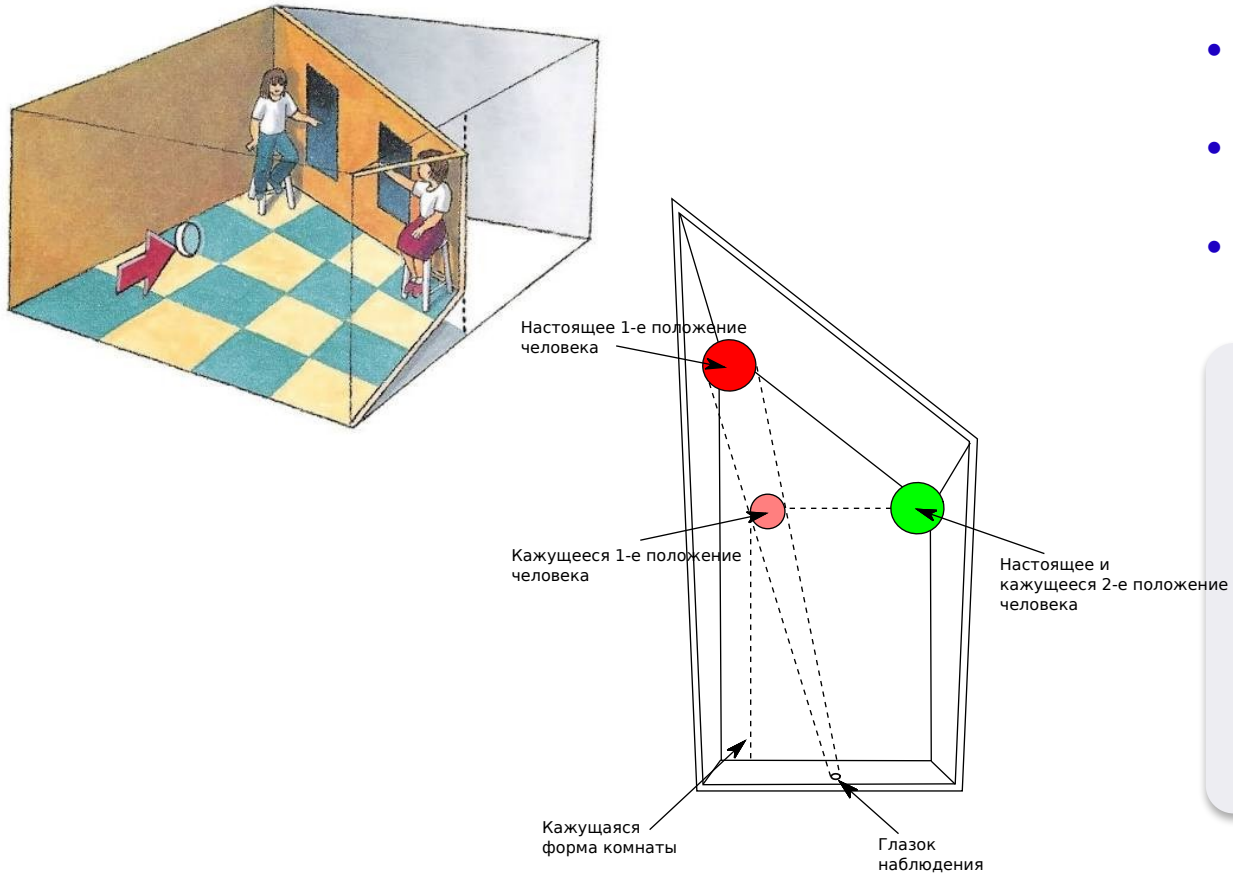
малый шум в пикселях двигает оценку

Это свойство самой задачи, а не нехватка вычислений

Вопрос при чтении статьи: чем именно этот метод доопределяет задачу?

трещина: Leonhard Lenz, CC0 · роспись на асфальте: Metanamorph, CC BY-SA 4.0





Kirill Borisenko, CC BY-SA 4.0, Wikimedia Commons · схема комнаты Эймса

- Пиксель фиксирует направление, но не расстояние
- Все точки одного луча дают один и тот же пиксель
- Маленькая кружка близко и большая далеко — одинаковая проекция

Комната Эймса

Проецируется так же, как обычная прямоугольная

Дальний угол вдвое дальше — но кажется, что на том же расстоянии

Мозг выбирает решение «комната прямоугольная» — и человек «меняет» рост

Снимается вторым видом, активным сенсором или приором





Buiobuione, CC BY-SA 4.0, Wikimedia Commons · Тадж-Махал

- Увеличим сцену и базу в s раз — картинки не изменятся
- Важно отношение размера к расстоянию, а оно то же

Семейство решений — группа подобия

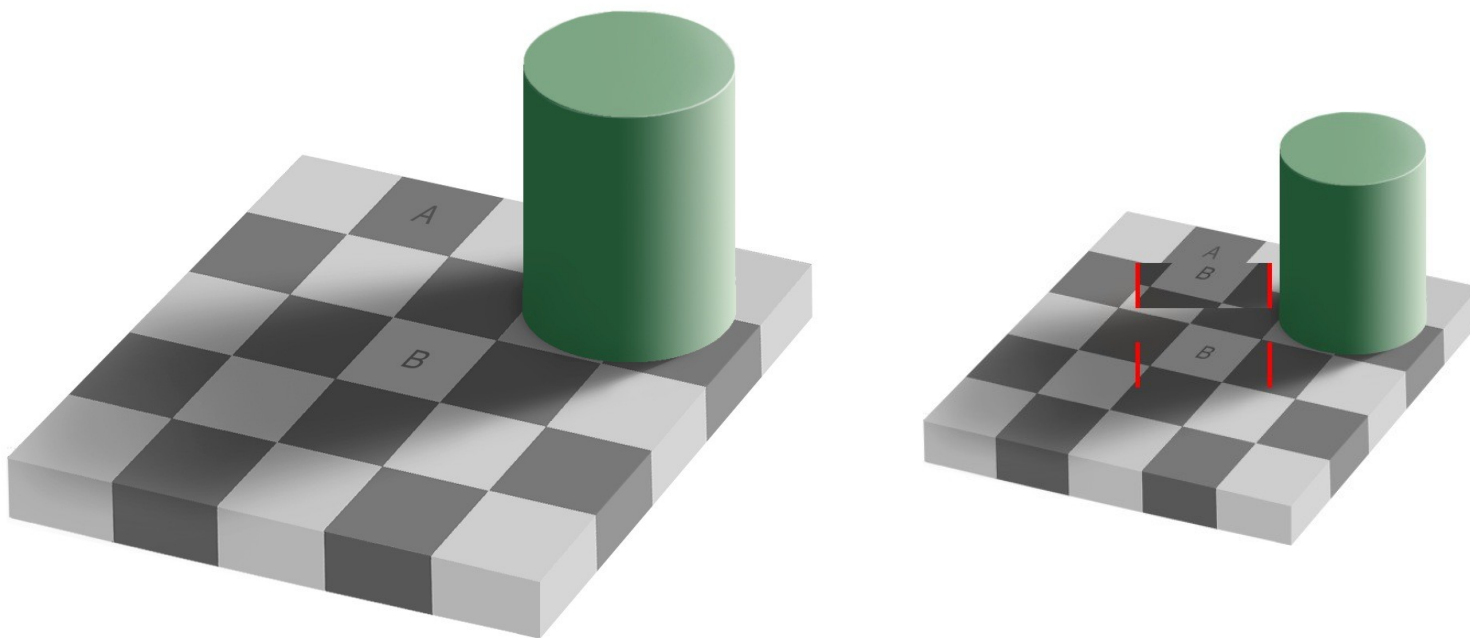
3 поворота + 3 сдвига + 1 масштаб = 7 степеней свободы
В лекции 5 это будет называться gauge freedom

Чем снимается

известный размер объекта в кадре
база стереопары
инерциальный датчик

Масштаб приходит только извне — из самих снимков его не достать





яркость = альбедо × освещённость

- одно число — два неизвестных
- тёмная поверхность при ярком свете и светлая при тусклом дают одно значение
- разделение — inverse rendering, лекция 8

оригинал Edward H. Adelson, файл Gustavb, Wikimedia Commons · клетки A и B одного оттенка

Бас-рельефная неоднозначность · Belhumeur, Kriegman, Yuille, 1999

При ламбертовой модели и ортографической проекции сжатие рельефа вдоль оси взгляда вместе с преобразованием освещения и альбедо даёт неотличимые изображения

По затенению при неизвестных освещении и альбедо форма восстанавливается лишь с точностью до этого семейства



А разве нейросети не восстанавливают по одному кадру?

Да, выдают. Но знание приходит из весов, а не из изображения



тот же механизм отказа, что у мозга: маска вогнутая, а приор
«лица выпуклые» побеждает измерение

Где отлично работает

визуализация, черновая модель, ассет — когда нужен правдоподобный результат

Где ломается

обратная сторона выдумана — свидетелей нет
масштаб угадан: сменится объектив — поедет оценка
вне распределения ошибается уверенно

Правдоподобный ответ $\neq$ измеренный ответ

нужны метры с известной точностью — нужны второй вид с известной базой или калиброванный сенсор





1. Больше наблюдений

второй кадр даёт второй луч —
пересечение определяет точку

плата: нужны позы камер и соответствия

лекции 4, 5, 12



2. Физические приоры

гладкость, ламбертовость,
манхэттенский мир

плата: мир может быть не таким

лекция 3



3. Выученные приоры

статистика мира,
извлечённая из данных

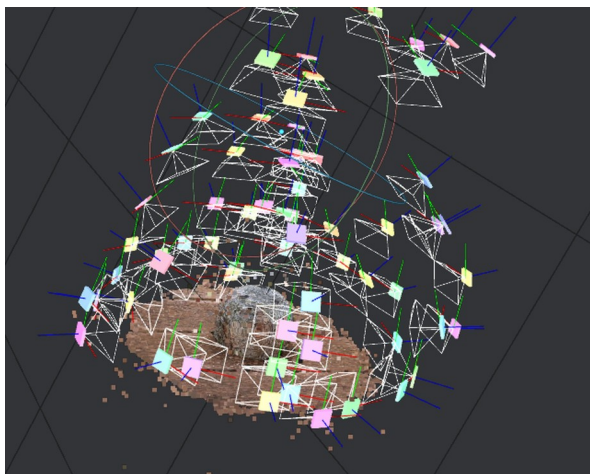
плата: зависит от обучающей выборки

лекции 9–11, 14

Ошибки у трёх источников разные — и это подсказывает, кто подвёл

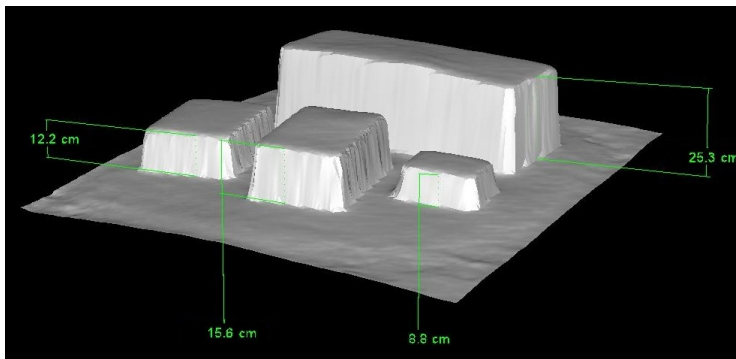
эпиполярные линии: .gs8, CC BY-SA 3.0 · Киото: Basile Morin, CC BY-SA 4.0 · монокулярная глубина: Kritik Soman, CC BY-SA 4.0





Многовидовая реконструкция

SfM и плотное стерео
по обычным фотографиям



Активный сенсор

RGB-D, лидар:
расстояние измерено



Синтетика

сцену задали числами —
геометрия известна точно

DUST3R — 8 наборов, $\approx 8,5$ млн пар · Depth Anything — 1,5 млн размеченных и 62 млн с псевдоразметкой

Выученный приор — это упакованная статистика ранее выполненных измерений

сеть выучивает те же признаки глубины, что и классическое зрение





NeRF: сцена — поле плотности и цвета



гауссовы сплаты: сцена — миллионы гауссиан

1. Рендерим гипотезу

прямая задача: по параметрам сцены
получаем изображение

2. Сравниваем со снимками

невязка по всем кадрам сразу

3. Двигаем параметры

градиент невязки по параметрам сцены,
и так до совпадения

Одна схема — разные представления сцены и разные рендереры

Прямую задачу разбираем в лекциях 6–8, обращаем её — в лекциях 9–12

NeRF: Jurdein, CC BY-SA 4.0 · гауссовы сплаты: Óscar Mirás, CC BY-SA 4.0



$$\theta^* = \arg \min_{\theta} \sum_k \|\mathcal{R}(\theta, \pi_k) - I_k\|^2 + \lambda \mathcal{P}(\theta)$$

$\mathcal{R}$ дифференцируемый рендерер —
прямая задача, лекции 6–8

I_k наблюдаемое изображение —
то, что сняла k-я камера

$\mathcal{P}$ регуляризатор —
тот самый приор

θ — параметры сцены, π — параметры камеры

Представление сцены обязано быть дифференцируемым

- иначе оно просто не встраивается в эту схему
- дифференцируемость — одна из осей, по которым мы сравниваем представления





ни одной из этих машин не существует: проход по латентному пространству генеративной модели формы

Реконструкция

$$\|\mathcal{R}(\theta, \pi_k) - I_k\|^2$$

невязка между рендером и реальным снимком

Генерация

$$D_\varphi(\mathcal{R}(\theta, \pi_k))$$

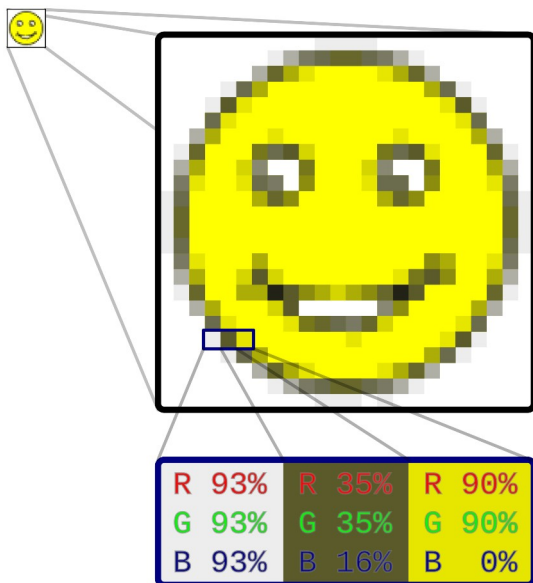
снимков нет: рендер оценивает обученная 2D-модель
(DreamFusion, 2022 — score distillation)

Наблюдений ноль — всю работу делает prior: это третий источник в чистом виде

без дифференцируемости некуда девать градиент, без свободы топологии не отрастить ручку

морфинг форм: Joël Jean Henry, CC BY-SA 4.0 · DreamFusion — Poole et al., 2022 (схема — лекции 9–10)





растр $W \times H \times C$: один формат
на сенсор, алгоритм и дисплей

В 2D повезло: представление одно

свёртка определена на решётке — отсюда все сети для изображений

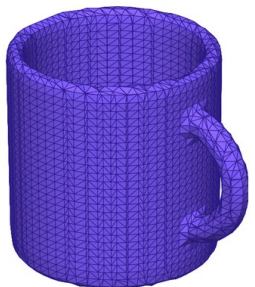
В 3D такого представления нет — и, похоже, не будет

- точка внутри или снаружи? одна структура
- где луч встречает поверхность? другая
- как плавно деформировать? третья
- как учить нейросеть? четвёртая

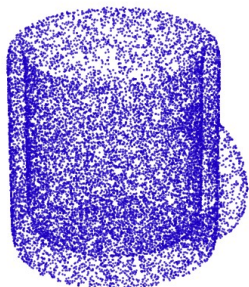
Представление — это структура данных плюс операции, которые она делает дешёвыми

Выбор представления — это выбор того, что будет дёшево, а что дорого

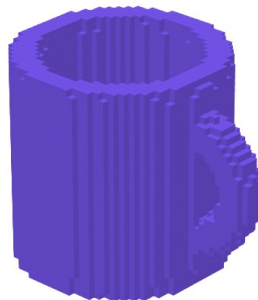




полигональная сетка
вершины и грани



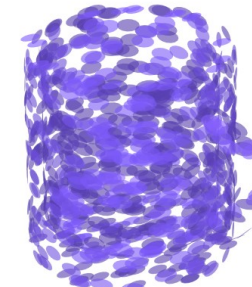
облако точек
множество без связей



воксели
регулярная 3D-сетка



неявная функция
поверхность = $\{ f = 0 \}$



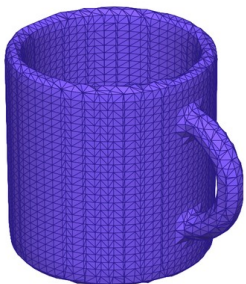
гауссовы сплаты
примитивы с объёмом

Разбираем каждое: что оно делает дешёвым и во что это обходится

и дифференцируемо ли оно — ось, которая появилась из схемы с рендерером



- поверхностные или объёмные — только граница тела или всё пространство
- явные или неявные — перечисляем точки или задаём функцию, чей нулевой уровень даёт поверхность



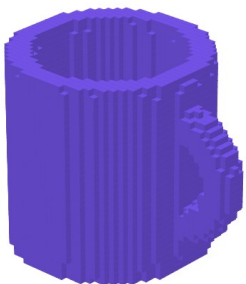
явно-поверхностные

облака точек, полигональные сетки,
параметрические поверхности



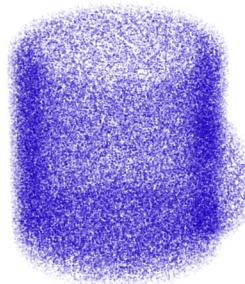
неявно-поверхностные

SDF, occupancy —
поверхность как нулевой уровень



объёмные дискретные

воксели, октодеревья, TSDF, MPI



объёмные непрерывные

поля плотности, поля излучения

Объёмное описывает поверхность, обратное — нет: поэтому идём от поверхности к объёму



$$\Phi(\mathbf{x}; \Theta)$$

поле, определённое во всех точках пространства, плюс параметры, которые где-то хранятся

Сигналы: что определено в точке

SDF — расстояние со знаком
оссирансу — внутри или снаружи
плотность, радиантность
глубина

Структуры: где живут параметры

воксельная сетка
октодерево, хэш-таблица
веса MLP
набор примитивов

Одна и та же оссирансу:

в вокселях — запрос $O(1)$, память $O(n^3)$; в MLP — память равна числу весов, запрос — полный проход сети

«Нейронное поле против вокселей» — это сравнение структур, а не сигналов

спрашивайте отдельно: какой сигнал и в чём он хранится

рамка «поле + параметры»: Xie et al., 2022



ось	что спрашиваем
лагранжево или эйлерово	информация едет вместе с точками или лежит в узлах сетки → динамические сцены
зависимость от ракурса	карта глубины видит одну сторону; сетка и неявная функция — объект целиком
память по разрешению	поверхность растёт как n^2 , объём — как n^3
топологическая свобода	может ли меняться число ручек в ходе оптимизации: у сеток сдвигом вершин нет, у неявных да
стоимость операций	внутри/снаружи · пересечение с лучом · ближайшая точка · сэмплирование · деформация · нормаль
дифференцируемость	есть ли градиент потерь по параметрам — без этого не встроить в схему с рендерером
пригодность для сетей	свёртки на регулярных, инвариантность к перестановкам на множествах, графы на сетках
редактируемость	возьмёт ли это в руки художник или инженер

Универсального представления нет — есть удачный выбор под задачу

оси из Botsch et al., Polygon Mesh Processing, гл. 1 и Tewari et al., STAR 2022



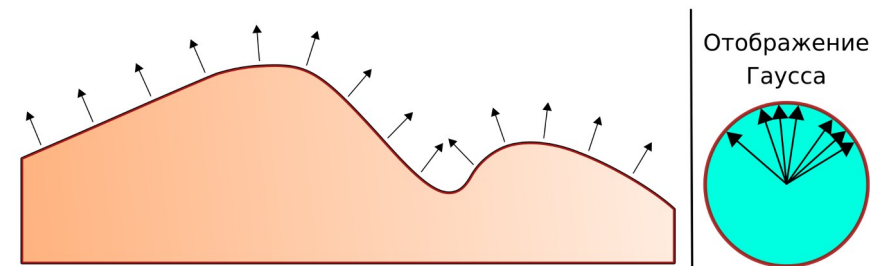


- вместе с цветом — RGB-D
- работают все инструменты обработки изображений, включая свёртки (лекция 11)

Нормаль — точка на сфере S^2 , а не три числа

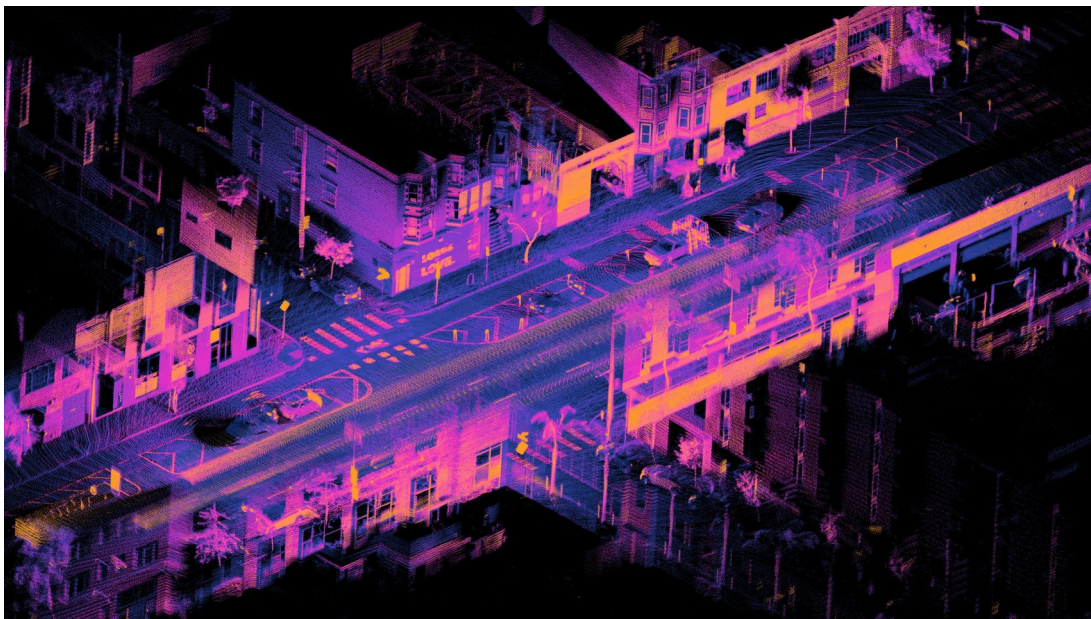
все нормали собираются на единичной сфере — отображение Гаусса

у сети единичность надо требовать явно; в лекции 2 то же с вращениями



отображение Гаусса: Erik Streb, CC BY-SA 3.0





лидар, времяпролётные камеры, структурированный свет,
а также стерео и SfM после обратной проекции

дёшево

получить и хранить: это сырой выход сенсора, без обработки

дорого

соседство: kd-дерево или радиусный поиск строятся заново

Отсортировать точки не выход: малое смещение меняет порядок скачком

лидарное облако: Daniel L. Lu, CC BY 4.0 (Ouster OS1-64, Сан-Франциско)

Порядок точек не несёт смысла

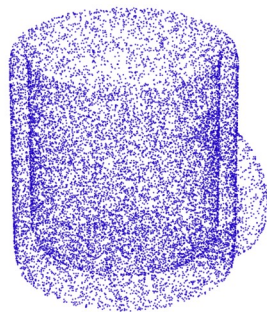
облако — множество, а не массив: ответ не должен меняться при перестановке точек

PointNet (Qi et al., 2017): симметричная агрегация — max-pooling по точкам

Связности нет

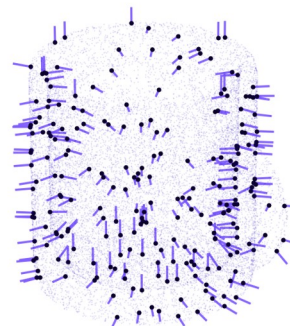
какие точки соседние — неизвестно; kd-дерево или радиусный поиск строятся заново на каждом шаге





облако точек $\{p_i\}$

16 000 точек; ни порядка, ни связей



ориентированное облако $\{(p_i, n_i)\}$

у каждой точки — единичная нормаль

- нормаль превращает точку в плоский диск — хватает, чтобы рендерить и реконструировать
- нормаль — из PCA окрестности: собственный вектор при наименьшем собственном значении

дёшево

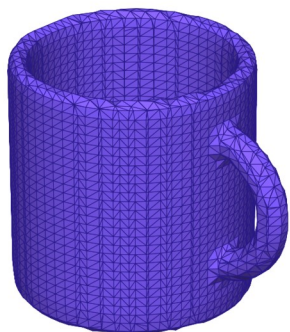
рендерить сплаттингом и реконструировать поверхность

дорого

согласовать знак: локально «наружу» не определено

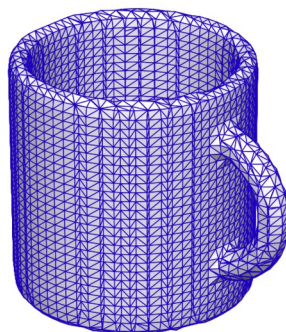
На семинаре 1: без согласования примерно половина нормалей смотрит внутрь





грани

10744 треугольников



вершины и рёбра

5372 вершин

- V — вершины, F — грани: упорядоченные наборы индексов
- почти всегда треугольники: три точки всегда лежат в одной плоскости, четыре — уже нет
- кусочно-линейная аппроксимация; точность растёт с числом вершин

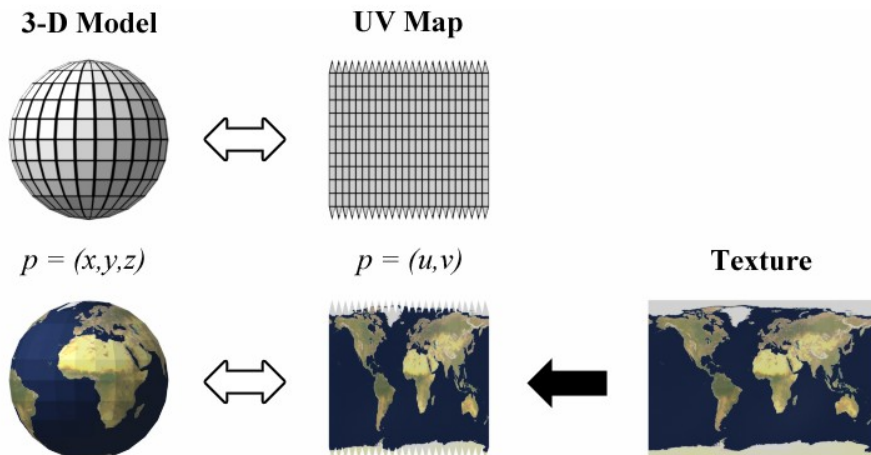
Порядок обхода вершин задаёт нормаль

отсюда отсечение задних граней в растеризации — и вывернутые модели при ошибках экспорта

Стандарт индустрии: в реальном времени всё рисуется треугольниками (лекция 6)

«наружу» у сетки не вычисляют, а читают: нормаль задана порядком перечисления вершин грани





форма — в сетке, цвет — в текстуре, связь — через UV

Многообразность (manifold)

каждое ребро принадлежит ровно двум граням, окрестность вершины устроена как диск

нужна большинству алгоритмов обработки; сканеры и marching cubes её дают, игровые движки — не всегда

дёшево

двигать вершины — на этом стоят модели тела и лица; сглаживать, упрощать, уплотнять

дорого

менять связность — неустойчиво

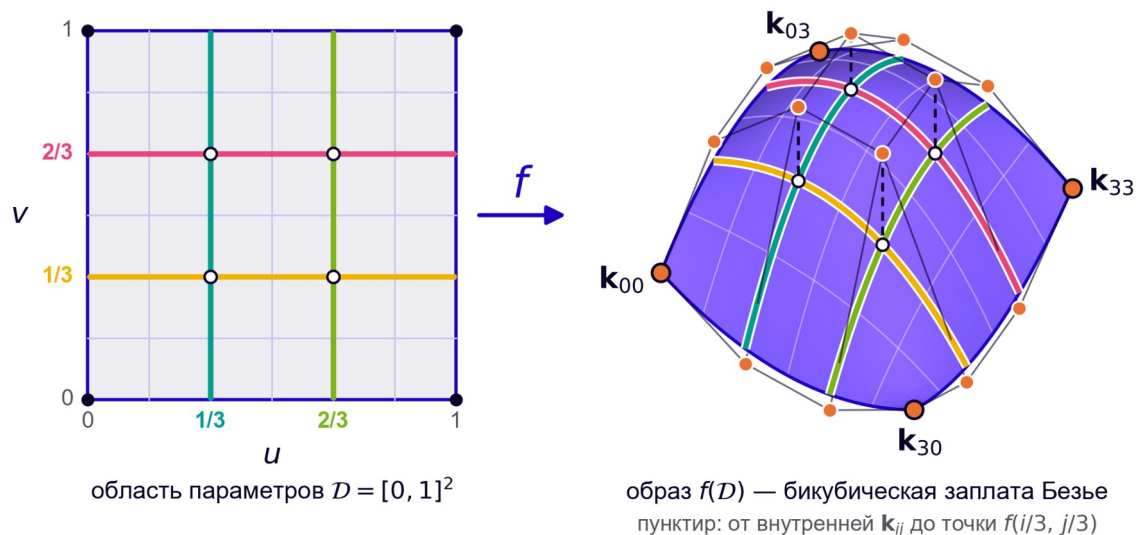
невозможно

сменить род сдвигом вершин без склеек, схлопываний и самопересечений

Топологическому ограничению посвящён отдельный раздел лекции

UV-развёртка: Tschmits, CC BY-SA 3.0





- поверхность — образ отображения $f: [0, 1]^2 \rightarrow \mathbb{R}^3$; при вложении образ — топологический диск
- Безье: сетка контрольных точек и полиномы Бернштейна; NURBS — язык промышленного CAD
- в CAD поверхность точная, а не аппроксимированная — треугольники только для экрана (тесселяция)

дёшево

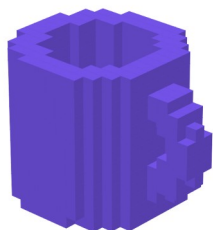
сэмплировать точки: случайные (u, v) ; равномерно по площади — с поправкой на $\|f_u \times f_v\|$

дорого

луч, ближайшая точка, «внутри/снаружи» — численно; у заплаты с краем внутренности нет

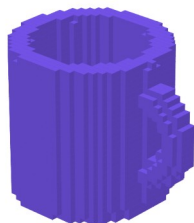
ту же идею реализуют нейросетью: AtlasNet, Groueix et al., 2018





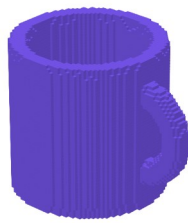
16³

занято 1185 из 4096 ячеек
плотный float32: 0.02 МБ



32³

занято 5155 из 32768 ячеек
плотный float32: 0.12 МБ



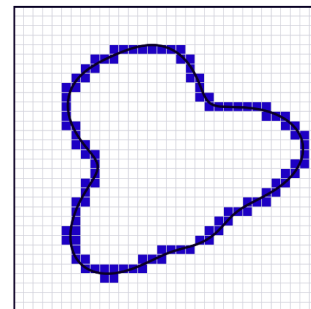
64³

занято 20719 из 262144 ячеек
плотный float32: 1.00 МБ

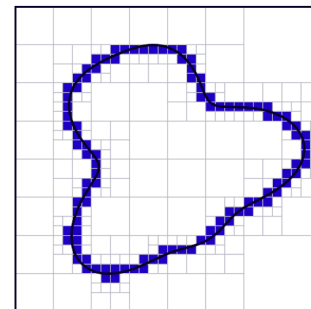


исходная сетка

90924 вершин,
181848 граней



плотная сетка: память $O(n^3)$, занято 10%
срез 32×32: хранятся все 1024 ячейки, у поверхности лишь 103



адаптивно: квадродерево (2D-аналог октодеревя)
295 ячеек вместо 1024: мелкие только у поверхности

- прямое обобщение растра: регулярная 3D-решётка, работают трёхмерные свёртки
- память $O(n^3)$, а поверхность двумерна: оболочка занимает 29 % ячеек при 16^3 , 8 % при 64^3 , около 1 % при 512^3 — и это полгигабайта на объект во float32
- октодеревя делит куб на восемь, пока ячейка пересекает поверхность: память падает до $O(n^2)$

дёшево

тест «внутри/снаружи», трассировка луча, свёртки прямо по сетке

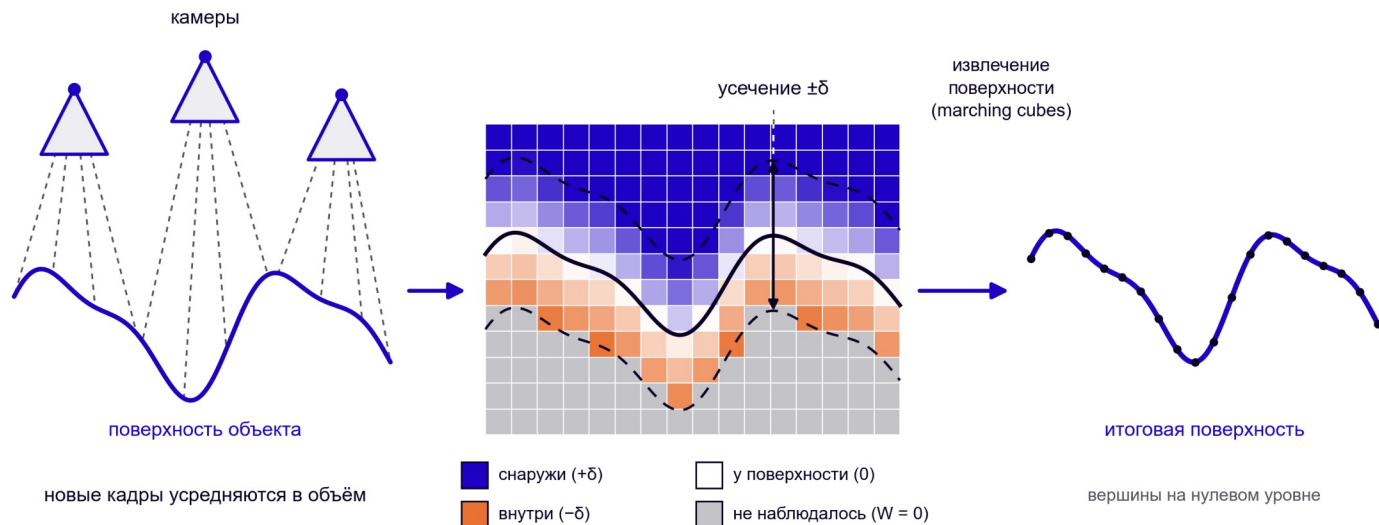
дорого

память по разрешению и любая деформация — значения живут в узлах

разреженные свёртки и хэш-сетки признаков: Instant-NGP, Müller et al., 2022 (лекция 10)



TSDF: как накапливают глубину



$$D \leftarrow \frac{WD + wd}{W + w}, W \leftarrow W + w$$

в ячейке — не занятость, а усечённое
знаковое расстояние и вес уверенности
усреднение подавляет шум сенсора
усечение $\pm\delta$ не даёт далёким поверхностям
мешать друг другу

структура та же, что у вокселей, —
изменился только сигнал

дёшево

влить новый кадр: одно взвешенное усреднение по ячейкам

дорого

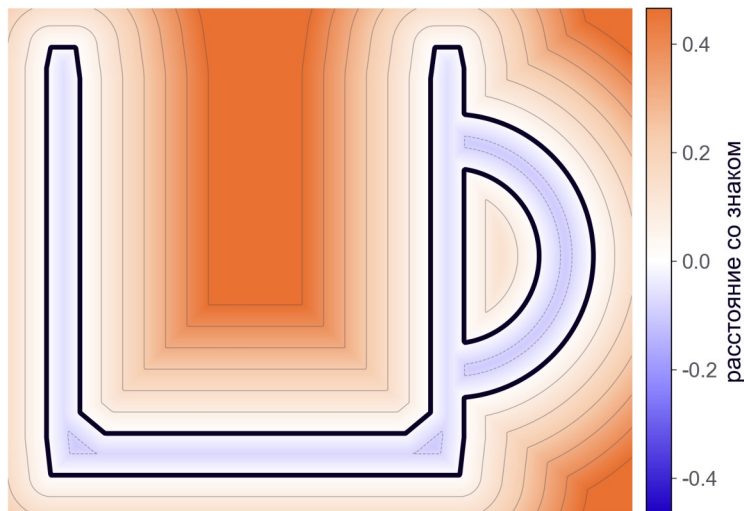
показать результат: нужно извлечение поверхности (marching cubes)

На TSDF построен KinectFusion и почти вся плотная реконструкция в реальном времени

Curless & Levoy, 1996; KinectFusion — Newcombe et al., 2011 (лекции 12 и 15)



вертикальный срез плоскостью, проходящей через ручку



$f = 0$ — поверхность $f < 0$ — внутри материала $f > 0$ — снаружи

$$\mathcal{S} = \{ \mathbf{p} \in \mathbb{R}^3 : f(\mathbf{p}) = 0 \}$$

$$\|\nabla f\| = 1$$

уравнение эйконала: длина градиента почти всюду единица, поэтому нормаль равна градиенту

- знак f — готовый тест «внутри/снаружи»
- градиент — нормаль, без всякой оценки по соседям
- топология меняется без ручной перестройки связности
- разрешение не фиксировано (кроме таблицы на сетке)

Бинарная занятость — это знак SDF, и ничего больше

классификация вместо регрессии: учить проще, но вдали от поверхности градиент не говорит, где она

дёшево

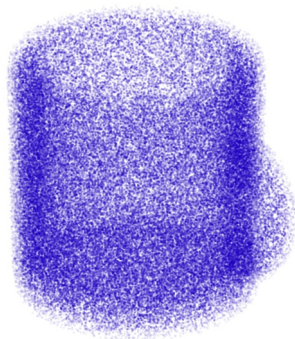
внутри/снаружи, нормали, смена топологии, опрос в любой точке

дорого

показать: нужно извлечение — marching cubes или трассировка лучей

DeepSDF — Park et al., 2019; Occupancy Networks — Mescheder et al., 2019; эйкональная регуляризация — IGR, Gropp et al., 2020





поле плотности $\sigma(\mathbf{p})$

нет поверхности — есть «сколько вещества» в каждой точке

$$C(\mathbf{r}) = \sum_i T_i (1 - e^{-\sigma_i \delta_i}) \mathbf{c}_i$$

- для дыма и тумана поверхности просто нет — есть плотность $\sigma(\mathbf{p})$ в каждой точке
- NeRF: (точка, направление) $\rightarrow$ (плотность, цвет); цвет зависит от взгляда — отсюда блики
- формула дифференцируема по всем σ и $\mathbf{c}$, а значит и по весам сети — ради этого представление и выбрано

дешево

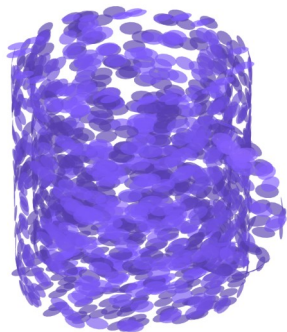
дифференцируемость всюду и обучение только по 2D-изображениям

дорого

поверхности нет: её извлечение — отдельная задача, геометрия шумная

NeRF существует ровно потому, что рендеринг сделали дифференцируемым





гауссовы сплаты

центр, ковариация, непрозрачность, цвет — у каждого

$$\Sigma_k = R_k S_k S_k^\top R_k^\top$$

- сцена — набор гауссиан: центр, ковариация, непрозрачность, цвет (сферические гармоники)
- разложение на вращение и ненулевой масштаб держит ковариацию положительно определённой при оптимизации
- рендеринг — растеризация и альфа-компози́тинг: структура как у NeRF, но на порядок быстрее
- число сплатов не фиксировано: гауссианы клонируются, делятся и удаляются

дёшево

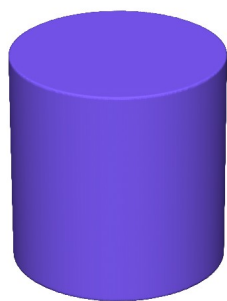
рендеринг и деформация: сплаты лагранжевы — переносят себя

дорого

явной поверхности нет — её извлечение отдельная задача

Ориентированное облако точек, у которого диски получили толщину и прозрачность





цилиндр

\



цилиндр поменьше

U



часть тора

=



кружка

CSG: тело как дерево булевых операций над примитивами

- B-пер — точные поверхности (плоскости, цилиндры, NURBS) плюс топология: как грани, рёбра и вершины связаны
- CSG — дерево булевых операций над примитивами: объединение, пересечение, вычитание
- история построения — эскизы и операции с параметрами и ссылками на грани; ядро CAD пересчитывает её в B-пер

дёшево

точность, компактность и правка детали изменением параметра операции

дорого

дифференцируемость и вход для нейросети — почти никак



Дерево построения

- 1 **Эскиз 1** плоскость XY
окружность **R = 40 мм**, центр (0, 0)
- 2 **Вытягивание** эскиз 1
высота H = 84 мм
- 3 **Оболочка**
толщина 6 мм, удалена **верхняя грань (2)**
- 4 **Эскиз 2** плоскость XZ
дуга радиуса 22 мм, центр на высоте H/2
и в 2 мм внутрь от **боковой грани (2)**;
концы в стенке, в 4 мм от этой грани
- 5 **Протяжка** траектория — эскиз 2
сечение — круг радиуса 5 мм
- 6 **Скругление**
радиус 10 мм, **ребро дна (2)**

1 · эскиз 1
R = 40 мм

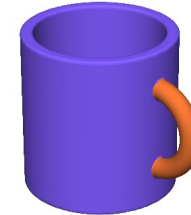
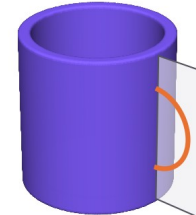
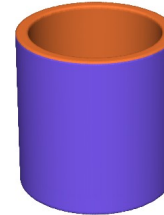
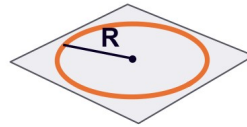
2 · вытягивание

3 · оболочка

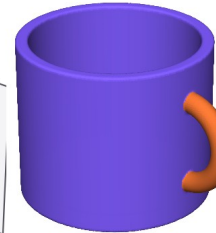
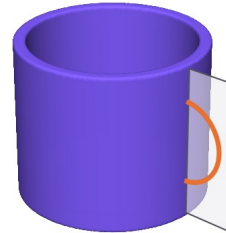
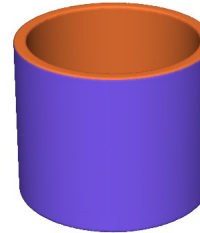
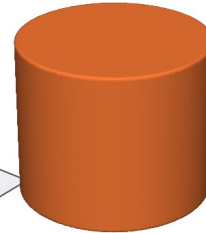
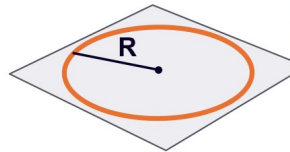
4 · эскиз 2

5 · протяжка

6 · скругление



R = 50 мм, остальные параметры те же: цепочка пересчитана



История построения: форма — результат пересчёта цепочки операций с параметрами; оранжевым — что создал шаг, синим — ссылки на элементы шага 2

Обратная разработка восстанавливает не форму, а замысел конструктора



Сводная таблица представлений

представление	память	внутри / снаружи	луч	деформация	смена топологии	вход для сети
карта глубины	$O(n^2)$	нет	тривиально	—	—	отлично (растр)
облако точек	$O(n^2)$	нет	сложно	легко	легко	инвариантность к перестановкам
полигональная сетка	$O(n^2)$	если замкнута	быстро	легко	сдвигом — нет	операции на графах
параметрическая	мало	трудно	численно	легко	вложение: нет	да
воксели	$O(n^3)$	тривиально	просто	тяжело	легко	отлично (3D-свёртки)
октодерево	$O(n^2)$	тривиально	просто	тяжело	легко	хорошо
SDF	любая	по знаку	sphere tracing	легко	бесплатно	отлично
поле плотности	любая	нет понятия	интеграл	легко	бесплатно	отлично
гауссовы сплаты	$O(n^2)$	нет	растеризация	легко	легко	хорошо
B-рег	мало	точно	аналитически	правкой граней	операцией	плохо
история построения	мало	после пересчёта	после пересчёта	параметром	вставкой операции	последовательность команд

- столкновения в физике — нужны «внутри/снаружи» и замкнутость: воксели, SDF, замкнутая сетка
- генератор мебели — нужна свобода топологии и дифференцируемость: неявные функции
- скан здания для замеров — нужны точность и компактность: облако точек или сетка

Таблицу читают не по строкам, а по задачам: ответ определяется задачей



$$\chi = V - E + F = 2 - 2g$$

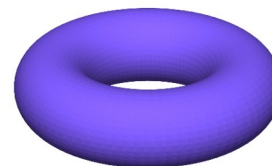
поверхность замкнута и ориентируема;
от триангуляции не зависит



сфера
род 0



кружка
род 1



тор
род 1

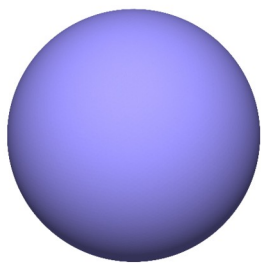
у кружки и тора одна дырка — непрерывной деформацией одно переходит в другое; сферу так не получить

- род g — число ручек: сфера 0, тор 1, двойной тор 2
- рёбра считают как уникальные пары вершин: наивная оценка $3F$ вдвое больше (проверите на семинаре 1)

Дырка — это сквозное отверстие, а не полость: полость чашки — вмятина на сфере



Род не меняется деформацией



шаблон: сфера
род 0



предел деформации: отверстие запаяно
род 0



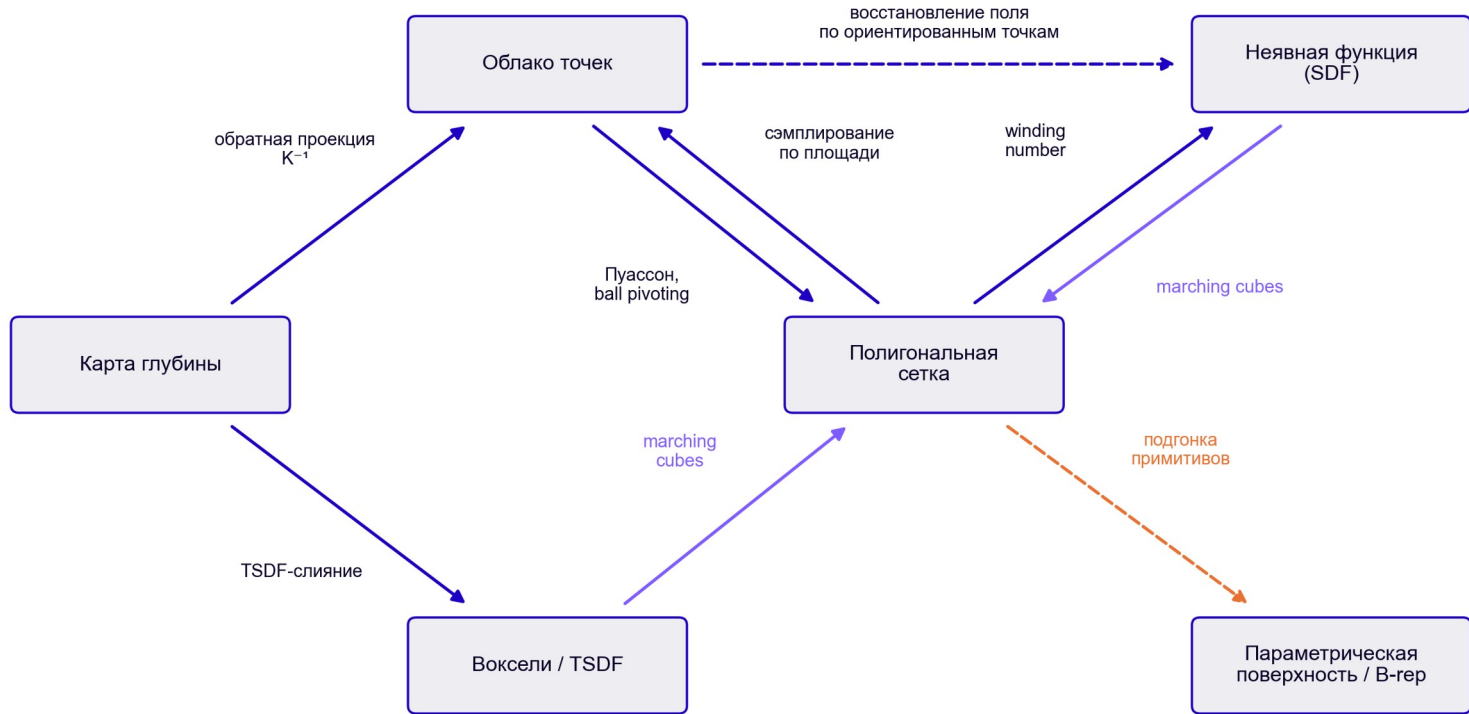
настоящая кружка
род 1

- методы «деформируй шаблон» стартуют со сферы, например Pixel2Mesh
- без склеек, схлопываний и самопересечений кружку с ручкой такой метод не построит никогда
- обходные пути — заплатки или шаблон нужного рода — неустойчивы или требуют знать топологию

У неявных функций род выпадает из значений f : f меняется непрерывно, род — скачком

поэтому генеративные методы формы построены на них, а не на сетках





работает плохо: надо восстановить замысел конструктора, а не форму

Что нужно

обратная проекция

калибровка K

TSDF-слияние

позы камер

Пуассон, ball pivoting

ориентированные нормали

подгонка примитивов

работает плохо

Для каждой пары — свой метод; общего решения не существует

но основных переходов немного, и все они хорошо изучены



тема этой лекции	где развивается
проекция и потеря размерности	лекция 3 — модель камеры, калибровка
нормали и вращения как точки на многообразии	лекция 2 — $SO(3)$, кватернионы, оптимизация
доопределение вторым видом	лекции 4–5 — эпиполярная геометрия, SfM
рендеринг сеток и лучей	лекции 6–8 — растеризация, трассировка, освещение
инверсия рендерера, NeRF и сплаты	лекции 9–10 — дифференцируемый рендеринг
карты глубины и 2.5D	лекция 11 — оценка глубины
TSDF, marching cubes, реконструкция по облаку	лекция 12 — многовидовая реконструкция
point map	лекция 14 и заключение — feed-forward реконструкция
октодеревья и карты	лекция 15 — SLAM

Каждая следующая лекция — выбор представления плюс способ доопределить задачу

представление берём из сводной таблицы, способ доопределить — из трёх источников: больше наблюдений, физические приоры, выученные приоры



- обратная задача зрения некорректна: одному изображению отвечает бесконечно много сцен
- это свойство самой задачи, а не нехватка вычислений — быстрый компьютер единственности не вернёт
- доопределяют тремя способами: больше наблюдений, физические приоры, выученные приоры
- современная форма решения — инверсия дифференцируемого рендерера, отсюда требование дифференцируемости
- универсального представления нет: это структура данных плюс операции, которые она делает дешёвыми
- род поверхности не меняется деформацией, и метрика об этом не предупредит
- переход между представлениями — свой метод на каждую пару, и у каждого своё входное требование

